

Domanda 1

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Considera la tabella con i formati di istruzione RISC-V mostrata nella figura seguente. Determina il tipo di istruzione e l'istruzione equivalente in linguaggio assembly per la seguente istruzione:

0000 0000 0010 0001 1110 0010 0011 0011

- (0,50 punti) Tipo di istruzione: R-type ✓
- (0,50 punti) Istruzione assembly: or x4, x3, x2 ✓

RISC-V Instruction Set

Core Instruction Formats

31	25	24	20	19	15	14	12	11	7	6	0	
funct7		rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]				rs1		funct3		rd		opcode		I-type
imm[11:5]		rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]		rs2		rs1		funct3		imm[4:1 11]		opcode		SB-type
imm[31:12]								rd		opcode		U-type
imm[20 10:1 11 19:12]								rd		opcode		UJ-type

RV32I Base Integer Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-ext
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	signed
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	unsigned
addi	ADD (Immediate)	I	0010011	0x0		rd = rs1 + imm	
xori	XOR (Immediate)	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR (Immediate)	I	0010011	0x6		rd = rs1 imm	
andi	AND (Immediate)	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[11:5]=0x00	rd = rs1 << imm[0:4]	zero-ext
srl	Shift Right Logical Imm	I	0010011	0x5	imm[11:5]=0x00	rd = rs1 >> imm[0:4]	msb-ext
srai	Shift Right Arith Imm	I	0010011	0x5	imm[11:5]=0x20	rd = rs1 >> imm[0:4]	signed
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	unsigned
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-ext
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-ext
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	
beq	Branch ==	SB	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	SB	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	SB	1100011	0x4		if(rs1 < rs2) PC += imm	signed
bge	Branch ≥	SB	1100011	0x5		if(rs1 ≥ rs2) PC += imm	signed
bltu	Branch < (U)	SB	1100011	0x6		if(rs1 < rs2) PC += imm	unsigned
bgeu	Branch ≥ (U)	SB	1100011	0x7		if(rs1 ≥ rs2) PC += imm	unsigned
jal	Jump And Link	UJ	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	
ecall	Environment Call	I	1110011	0x0	imm=0x0	Context switch to OS	
ebreak	Environment Break	I	1110011	0x0	imm=0x1	Context switch to debugger	
fence	Memory fence	FENCE	0001111	0x0		Order mem/io accesses	

Domanda 2

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Assumendo una architettura a 32 bit, supponiamo che i registri **x5** e **x6** contengano i valori **0x00000000** e **0xFFFFFFFF** rispettivamente. Determinare quale sarà il contenuto di **x30** dopo l'esecuzione di un'istruzione che somma i contenuti di **x5** e **x6**.

x30 = ✓

Il risultato in **x30** è corretto, o si è verificato un overflow? ✓

Si supponga nuovamente che i registri **x5** e **x6** contengano i valori riportati sopra (**0x00000000** e **0xFFFFFFFF**). Determinare quale sarà il contenuto di **x30** dopo l'esecuzione di un'istruzione che sottrae il contenuto di **x6** da **x5**.

x30 = ✓

Il risultato in **x30** è corretto o si è verificato un overflow? ✓

Domanda 3

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri che la memoria principale contenga i valori riportati nella figura, dove il numero a sinistra della cella rappresenta l'indirizzo della cella.

Memoria

7	0x87
6	0x86
5	0x85
4	0x84
3	0x83
2	0x82
1	0x81
0	0x80

- Se il contenuto memorizzato dal registro **x4** fosse **0xFFFFFFFF** e se quello del registro **x5** fosse **0x00000000** quale sarebbe il valore (4 byte in esadecimale) del registro **x4** se il processore eseguisse l'istruzione **lhu x4, 4(x5)**?

✓

- Se il contenuto memorizzato dal registro **x4** fosse **0x00112233** e se quello del registro **x5** fosse **0x00000000** quale sarebbe il valore del byte all'indirizzo 4 se il processore eseguisse l'istruzione **sh x4, 0(x5)**?

✓

Riportare i risultati in esadecimale, senza alcun carattere di spaziatura (ad esempio: 0xFFFFFFFF o 0xFF).

Domanda 4

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il compilatore:

Scegli un'alternativa:

- ☒ a. traduce il codice sorgente in codice oggetto, generando file intermedi ✓
- ☐ b. risolve il problema dei riferimenti esterni ad un modulo oggetto
- ☐ c. traduce il codice sorgente in codice macchina eseguibile direttamente dal processore
- ☐ d. risolve il problema del trasferimento in memoria centrale di un eseguibile

La risposta corretta è: traduce il codice sorgente in codice oggetto, generando file intermedi

Domanda 5

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si considerino due diverse implementazioni dello stesso set di istruzioni che può essere suddiviso in quattro classi: A, B, C e D, basate sui loro CPI (Cicli Per Istruzione).

Il processore P1 ha una frequenza di clock di 4 GHz e CPI di 10 (istruzioni di classe A), 10 (istruzioni di classe B), 2 (istruzioni di classe C) e 2 (istruzioni di classe D).

Il processore P2 ha una frequenza di clock di 2 GHz e CPI di 2 (istruzioni di classe A), 1 (istruzioni di classe B), 1 (istruzioni di classe C) e 1 (istruzioni di classe D).

Si consideri un programma costituito da 10000 istruzioni così suddivise: 30% di classe A, 20% di classe B, 40% di classe C e 10% di classe D.

- [0.3 punti] Determinare il numero di cicli di clock richiesti da P1 per eseguire il programma. ✓
- [0.3 punti] Determinare il numero di cicli di clock richiesti da P2 per eseguire il programma. ✓
- [0.4 punti] Quale processore esegue il programma più velocemente? ✓

Domanda 6

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Considerando un processore RISC-V a 32 bit, si supponga che il registro PC abbia il valore **0x00400008** (in esadecimale), e che l'istruzione corrente sia la prima istruzione riportata nel frammento di codice RISC-V di seguito.

Determinare l'indirizzo che sarà presente nel PC dopo l'esecuzione dell'istruzione **blt t0, zero, cont1**

```
addi t0, zero, -4
addi t1, zero, -4
xor t0, t0, t1
blt t0, zero, cont1
addi t0, zero, 0
jal zero, cont0
```

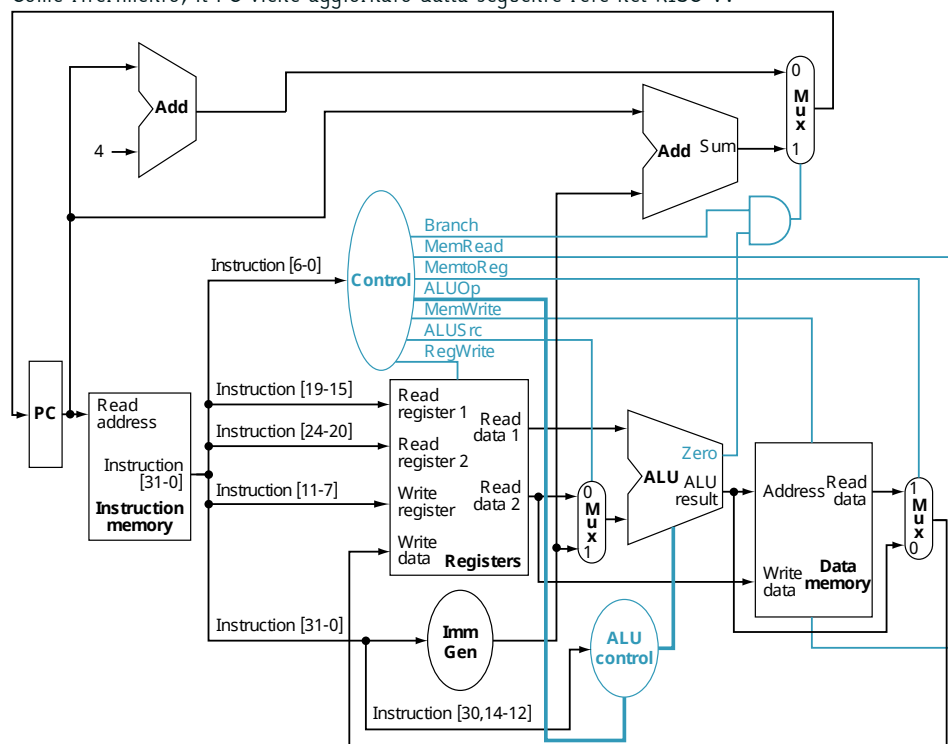
cont1:

```
addi t0, zero, 1
```

cont0:

```
nop
```

Come riferimento, il PC viene aggiornato dalla seguente rete nel RISC-V:



Scegli un'alternativa:

- ☐ a. 0x0040001C
- ☐ b. 0x00400014
- ☒ c. 0x00400018 ✓
- ☐ d. 0x00400010
- ☐ e. 0x00400020

La risposta corretta è: **0x00400018**

Domanda 7

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Di seguito viene riportata una sequenza di nove richieste di dati a una memoria cache a **mappatura diretta** di otto blocchi, inizialmente vuota. Gli **indirizzi dei blocchi** nelle richieste sono espressi in **binario a 5 bit**.

Numero della richiesta - Indirizzo in binario:

1. 00000
2. 01000
3. 00000
4. 01000
5. 10000
6. 00000
7. 11000
8. 10011
9. 11000

[0.5 punti] Completare la tabella seguente riportando lo stato della cache **dopo le nove richieste**, indicando per ogni blocco il **tag** del blocco memorizzato, o se il blocco è ancora **non valido**.

• Linea/line: 000	Tag:	11	✓
• Linea/line: 001	Tag:	Non valido/Invalid	✓
• Linea/line: 010	Tag:	Non valido/Invalid	✓
• Linea/line: 011	Tag:	10	✓
• Linea/line: 100	Tag:	Non valido/Invalid	✓
• Linea/line: 101	Tag:	Non valido/Invalid	✓
• Linea/line: 110	Tag:	Non valido/Invalid	✓
• Linea/line: 111	Tag:	Non valido/Invalid	✓

[0.5 punti] Quanti sono stati i **hit** e i **miss** nella cache?

- Hit: ✓
- Miss: ✓

Domanda 8

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Due compilatori diversi hanno tradotto il seguente frammento di codice C:

```
if (!(a || b)) {  
    x = 1;  
}
```

Assumi che **a** sia in **t0**, **b** sia in **t1**, e **x** sia in **t2**.

Compilatore A:

```
1      or t3, t0, t1  
2      beqz t3, xset  
3      j skip  
4 xset: li t2, 1  
5 skip: nop
```

Compilatore B (incompleto):

```
1      _____  
2      beqz t1, xset  
3      j skip  
4 xset: li t2, 1  
5 skip: nop
```

Quale istruzione manca nel codice del Compilatore B (linea 1) per renderlo logicamente equivalente a quello del Compilatore A?

Scegli un'alternativa:

- ☒ a. bnez t0, skip ✓
- ☐ b. bne t0, t1, skip
- ☐ c. beqz t0, skip
- ☐ d. and t3, t0, t1
- ☐ e. beq t0, t1, xset
- ☐ f. or t3, t0, t1

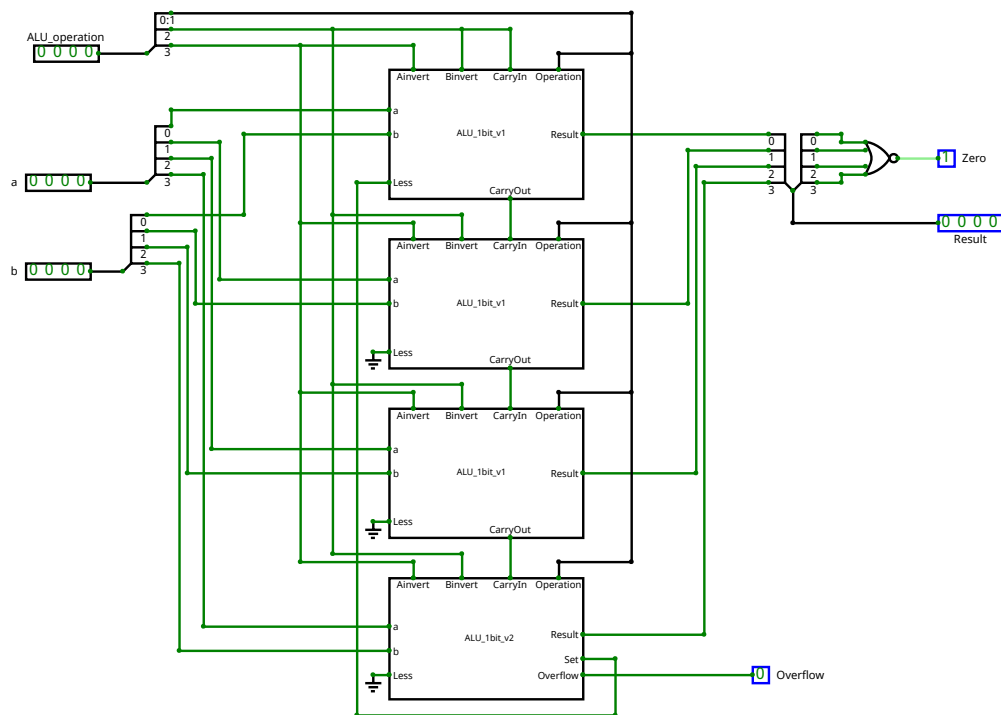
La risposta corretta è: bnez t0, skip

Domanda 9

Risposta corretta

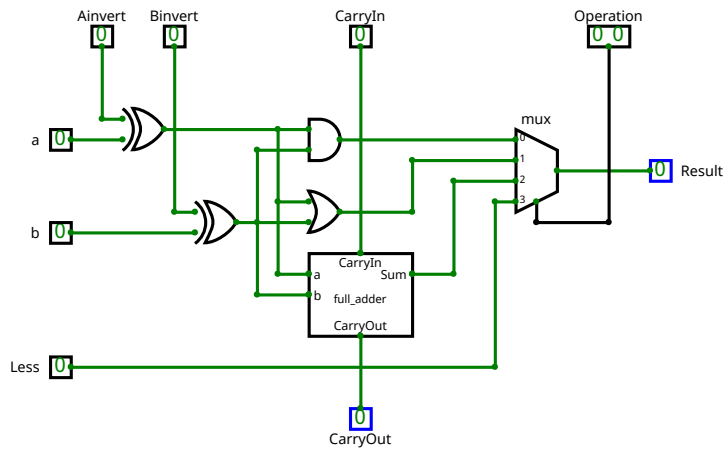
Punteggio ottenuto 1,00 su 1,00

Si consideri una ALU a 4 bit come realizzata in laboratorio.

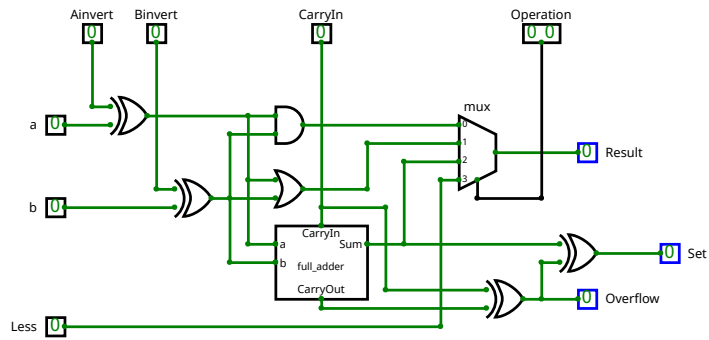


Questa ALU a 4 bit è composta da 3 ALU a 1 bit di "tipo 1" e 1 ALU a 1 bit di "tipo 2", che sono realizzate come nelle figure seguenti:

ALU_1bit_v1



ALU_1bit_v2



Assumendo che siano portati in ingresso i seguenti valori binari:

- a = 1111
- b = 0010
- ALU_operation = 0010

Determinare le uscite dall'ALU.

- (0.50 punti) Result = ✓
- (0.25 punti) Zero = ✓
- (0.25 punti) Overflow = ✓

Scrivere il valore **Result** in uscita dall'ALU come stringa di 4 bit, es. 1010.

Domanda 10

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Nel modello della macchina di Von Neumann NON è previsto che:

Scegli un'alternativa:

- ☐ a. le istruzioni debbano essere prelevate, decodificate ed eseguite
- ☐ b. dati ed istruzioni risiedano entrambi in memoria centrale
- ☐ c. dati ed istruzioni siano entrambi codificati con sequenze di bit
- ☒ d. i dati debbano essere prelevati, decodificati ed elaborati ✓

La risposta corretta è: i dati debbano essere prelevati, decodificati ed elaborati

Domanda 11

Risposta corretta

Punteggio ottenuto 4,00 su 4,00

Scrivere una procedura chiamata `dotproduct(arr1, arr2, size)` che calcola il prodotto scalare di due array di interi. Le variabili `arr1` e `arr2` sono array di word, e `size` è il numero di elementi in ciascun array.

Il prodotto scalare è calcolato come: $arr1[0]*arr2[0] + arr1[1]*arr2[1] + \dots + arr1[size-1]*arr2[size-1]$

Il valore di ritorno deve essere lasciato nel registro `a0`.

Il seguente codice in C implementa `dotproduct` (convertirlo in RISC-V):

```
// int (in C) è equivalente a word (in RISC-V)
int dotproduct(int arr1[], int arr2[], int size) {
    int result = 0;
    for (int i = 0; i < size; i++) {
        result += arr1[i] * arr2[i];
    }
    return result;
}
```

Attenzione:

- Incollare solo la funzione `dotproduct` (in RISC-V) nel campo sottostante
- Prestare attenzione alle convenzioni di chiamata!
- Utilizzare il seguente codice `main` per lo sviluppo e il debug nel simulatore

```
.globl _start
.data
    arr1: .word 2, 3, -1, 4, 5
    arr2: .word 1, -2, 3, 0, 2
    size: .word 5

.text
_start:
    # chiama dotproduct
    la  a0, arr1
    la  a1, arr2
    la  a2, size
    lw  a2, 0(a2)
    jal ra, dotproduct

    li  a7, 10
    ecall
```

```
*****
# completare la funzione dotproduct nel campo sottostante
```

	Test	Atteso	Ottenuto	
✓	<pre>.section .data arr1: .word 2, 3, -1, 4, 5 arr2: .word 1, -2, 3, 0, 2 size: .word 5</pre>	a0: 3	a0: 3	✓

Tutti i test sono stati superati ✓

▼ Visualizza/Nascondi soluzione dell'autore (Asm)

```

1 |
2 ▼ #####
3 # Procedure dotproduct(arr1, arr2, size)
4 # a0 -> address of arr1
5 # a1 -> address of arr2
6 # a2 -> size of arrays
7 # return dot product of the two arrays
8 #####
9 ▼ dotproduct:
10     li    t0, 0 # i
11     li    t1, 0 # result
12
13 ▼ loop_start:
14     # check if i < size
15     bge   t0, a2, loop_end
16
17     # load arr1[i] into t2
18     slli  t2, t0, 2
19     add   t2, t2, a0
20     lw    t2, 0(t2)
21
22     # load arr2[i] into t3
23     slli  t3, t0, 2
24     add   t3, t3, a1
25     lw    t3, 0(t3)
26
27     # multiply arr1[i] * arr2[i] and add to result
28     mul   t2, t2, t3
29     add   t1, t1, t2
30
31     # increment i
32     addi  t0, t0, 1
33
34     j     loop_start
35
36 ▼ loop_end:
37     mv    a0, t1
38     ret
39
40
41

```

⌂

Risposta corretta

Punteggio di questo invio: 4,00/4,00.

Commento:

ok

Domanda **12**

Risposta corretta

Punteggio ottenuto 4,00 su 4,00

Scrivere una procedura chiamata `bothshort(str1, str2, N)` che riceve due stringhe `str1` e `str2` e un intero `N` e restituisce:

- 1 se entrambe le stringhe hanno lunghezza minore o uguale a `N`
- 0 altrimenti

La funzione `bothshort` deve utilizzare la funzione `strlen` che riceve una stringa e restituisce la sua lunghezza. Soluzioni che non utilizzano la funzione `strlen` saranno considerate errate.

`strlen` è già implementata in RISC-V e non deve essere riscritta. Il valore di ritorno deve essere lasciato nel registro `a0`. Il seguente codice in C implementa `bothshort` (convertirlo in RISC-V):

```
// int (in C) è equivalente a word (in RISC-V)
int bothshort(char *str1, char *str2, int N) {
    if (strlen(str1) <= N && strlen(str2) <= N)
        return 1;
    else
        return 0;
}
```

Attenzione:

- Incollare solo la funzione `bothshort` (in RISC-V) nel campo sottostante
- Prestare attenzione alle convenzioni di chiamata!
- Utilizzare il seguente codice main per lo sviluppo e il debug nel simulatore

```
.globl _start
.data
    str1: .string "dog"
    str2: .string "cat"
    N:    .word 4

.text
_start:
    # chiama bothshort
    la    a0, str1
    la    a1, str2
    la    a2, N
    lw    a2, 0(a2)
    jal   ra, bothshort

    li    a7, 10
    ecall

#####
# strlen
# a0 -> str
# ritorna la lunghezza della stringa in a0
#####
strlen:
    mv    t0, a0
sr1:    lbu    t1, 0(t0)
        beqz   t1, sr2
        addi   t0, t0, 1
        j      sr1
sr2:    sub    a0, t0, a0
        ret

#####
# completare la funzione bothshort nel campo sottostante
```

	Test	Atteso	Ottenuto	
✓	<pre>.section .data str1: .string "dog" str2: .string "cat" N: .word 4</pre>	a0: 1	a0: 1	✓

Tutti i test sono stati superati ✓

▼ Visualizza/Nascondi soluzione dell'autore (Asm)

```

1
2 ▼ #####
3 # Procedure bothshort(str1, str2, N)
4 # a0 -> str1
5 # a1 -> str2
6 # a2 -> N
7 # return: 1 if both strings have length <= N, 0 otherwise
8 #####
9 ▼ bothshort:
10     addi sp, sp, -16
11     sw   ra, 0(sp)
12     sw   s1, 4(sp)
13     sw   s2, 8(sp)
14     sw   s3, 12(sp)
15
16     mv   s1, a1
17     mv   s2, a2
18
19     jal  ra, strlen # get length of str1
20     mv   s3, a0
21
22     mv   a0, s1
23     jal  ra, strlen # get length of str2
24
25     bgt  s3, s2, bothshort_ret0
26     bgt  a0, s2, bothshort_ret0
27     li   a0, 1
28     j    bothshort_end
29 ▼ bothshort_ret0:
30     li   a0, 0
31 ▼ bothshort_end:
32     lw   ra, 0(sp)
33     lw   s1, 4(sp)
34     lw   s2, 8(sp)

```